# OS Jackfruit – Container Runtime with Kernel Memory Enforcement

## 1. Team Information


- Vaibhav R (SRN: PES2UG24CS664)
- Vikas N (SRN: PES2UG24CS668)


---

## 2. Build, Load, and Run Instructions

### 🔹 Build the Project

```
make
```

---

### 🔹 Load Kernel Module

```
sudo insmod monitor.ko
```

Verify device:

```
ls -l /dev/container_monitor
```

---

### 🔹 Start Supervisor

```
sudo ./engine supervisor ./rootfs-base
```

---

### 🔹 Prepare Container Filesystems

```
cp -a ./rootfs-base ./rootfs-alpha
cp -a ./rootfs-base ./rootfs-beta

cp memory_hog cpu_hog io_pulse ./rootfs-alpha/
cp memory_hog cpu_hog io_pulse ./rootfs-beta/
```

---

- Start Containers

```
sudo ./engine start c1 ./rootfs-alpha ./memory_hog
sudo ./engine start c2 ./rootfs-beta ./cpu_hog
```

---

- List Containers

```
sudo ./engine ps
```

---

- View Logs

```
sudo ./engine logs c1
```

---

- Stop Containers

```
sudo ./engine stop c1
sudo ./engine stop c2
```

---

- View Kernel Logs

```
dmesg | tail
```

---

- Unload Module

```
sudo rmmod monitor
```

---

## 3. Demo with Screenshots

### 3.1 Multi-container Supervision

```
vaibhav-pc@vaibhav-pc-VirtualBox:~/Desktop/OS-Jackfruit/boilerplate$ sudo ./engine supe
rvisor ./rootfs-base
[sudo] password for vaibhav-pc:
[supervisor] control socket: /tmp/mini_runtime.sock
[supervisor] logging thread started
[supervisor] ready. base-rootfs=./rootfs-base
[supervisor] launched container id=c1 pid=4723
[supervisor] launched container id=c2 pid=4757
```

Two containers running under a single supervisor process.

---

### 3.2 Metadata Tracking

```
vaibhav-pc@vaibhav-pc-VirtualBox:~/Desktop/OS-Jackfruit/boilerplate$ sudo ./engine ps
c2                pid=4757   state=exited     soft=40MiB hard=64MiB log=logs/c2.log
c1                pid=4723   state=killed     soft=40MiB hard=64MiB log=logs/c1.log
```

Output of `engine ps` showing container metadata such as PID, state, and limits.

---

### 3.3 Bounded-buffer Logging

```
vaibhav-pc@vaibhav-pc-VirtualBox:~/Desktop/OS-Jackfruit/boilerplate$ sudo ./engine logs
 c1
allocation=1 chunk=8MB total=8MB
allocation=2 chunk=8MB total=16MB
allocation=3 chunk=8MB total=24MB
allocation=4 chunk=8MB total=32MB
allocation=5 chunk=8MB total=40MB
allocation=6 chunk=8MB total=48MB
allocation=7 chunk=8MB total=56MB
```

Container output captured via bounded-buffer logging pipeline demonstrating producer-consumer behavior.

---

### 3.4 CLI and IPC

```
Files v-pc@vaibhav-pc-VirtualBox:~/Desktop/OS-Jackfruit/boilerplate$ sudo ./engine star
t c1 ./rootfs-alpha ./memory_hog
[sudo] password for vaibhav-pc:
started container 'c1' pid=4723
```

```
vaibhav-pc@vaibhav-pc-VirtualBox:~/Desktop/OS-Jackfruit/boilerplate$ sudo ./engine supe
rvisor ./rootfs-base
[sudo] password for vaibhav-pc:
[supervisor] control socket: /tmp/mini_runtime.sock
[supervisor] logging thread started
[supervisor] ready. base-rootfs=./rootfs-base
[supervisor] launched container id=c1 pid=4723
```

CLI command issued and supervisor responding via UNIX socket IPC.

---

### 3.5 Soft-limit Warning

```
vaibhav-pc@vaibhav-pc-VirtualBox:~/Desktop/OS-Jackfruit/boilerplate$ sudo dmesg -w
[ 6328.928928] [container_monitor] Registering container=c1 pid=4723 soft=41943040 hard
=67108864
[ 6333.594520] [container_monitor] SOFT LIMIT container=c1 pid=4723 rss=42541056 limit=
41943040
[ 6336.668515] [container_monitor] HARD LIMIT container=c1 pid=4723 rss=67706880 limit=
67108864
[ 6336.678976] [container_monitor] Unregister request container=c1 pid=4723
[ 6752.307311] [container_monitor] Registering container=c2 pid=4757 soft=41943040 hard
=67108864
[ 6762.056015] [container_monitor] Unregister request container=c2 pid=4757
```

Kernel logs showing soft memory limit warning.

---

## 3.6 Hard-limit Enforcement

```
vaibhav-pc@vaibhav-pc-VirtualBox:~/Desktop/OS-Jackfruit/boilerplate$ sudo dmesg -w
[ 6328.928928] [container_monitor] Registering container=c1 pid=4723 soft=41943040 hard
=67108864
[ 6333.594520] [container_monitor] SOFT LIMIT container=c1 pid=4723 rss=42541056 limit=
41943040
[ 6336.668515] [container_monitor] HARD LIMIT container=c1 pid=4723 rss=67706880 limit=
67108864
[ 6336.678976] [container_monitor] Unregister request container=c1 pid=4723
[ 6752.307311] [container_monitor] Registering container=c2 pid=4757 soft=41943040 hard
=67108864
[ 6762.056015] [container_monitor] Unregister request container=c2 pid=4757
```

Kernel logs showing process termination after exceeding hard limit.

---

## 3.7 Scheduling Experiment

```
vaibhav-pc@vaibhav-pc-VirtualBox:~/Desktop/OS-Jackfruit/boilerplate$ sudo ./engine star
t prio_high ./rootfs-alpha ./cpu_hog --nice -20 && sudo ./engine start prio_low ./rootf
s-beta ./cpu_hog --nice 19 && top
started container 'prio_high' pid=4896
started container 'prio_low' pid=4905
```

```
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 4896 root       0 -20    1012    736    680 R 100.0   0.0   0:01.63 cpu_hog
 4905 root      39  19    1012    732    680 R 100.0   0.0   0:01.57 cpu_hog
 1723 vaibhav+  20   0 4901636 482824 152156 R  13.0  12.0   6:01.15 gnome-shell
 3155 vaibhav+  20   0  702584  60332  46140 S   2.0   1.5   0:23.89 gnome-terminal-
 4908 vaibhav+  20   0   14532   5720   3496 R   1.0   0.1   0:00.02 top
 [illegible] root      20   0   23260  14656   9880 S   0.0   0.4   0:04.75 systemd
```

Settings

Processes with different nice values showing different CPU scheduling priorities.

---

## 3.8 Clean Teardown

```
vaibhav-pc@vaibhav-pc-VirtualBox:~/Desktop/OS-Jackfruit/boilerplate$ sudo ./engine supe
rvisor ./rootfs-base
[sudo] password for vaibhav-pc:
[supervisor] control socket: /tmp/mini_runtime.sock
[supervisor] logging thread started
[supervisor] ready. base-rootfs=./rootfs-base
[supervisor] launched container id=c1 pid=4723
[supervisor] launched container id=c2 pid=4757
[supervisor] launched container id=prio_high pid=4828
[supervisor] launched container id=prio_low pid=4840
[supervisor] launched container id=prio_high pid=4878
[supervisor] launched container id=prio_low pid=4886
[supervisor] launched container id=prio_high pid=4896
[supervisor] launched container id=prio_low pid=4905
^C[supervisor] shutting down
[supervisor] logging_thread: exiting, buffer drained
[supervisor] logger thread joined
[supervisor] clean exit
vaibhav-pc@vaibhav-pc-VirtualBox:~/Desktop/OS-Jackfruit/boilerplate$ ps aux | grep engi
ne
vaibhav+    2267  0.0  0.1 236776  7676 ?        Sl   09:33   0:03 /usr/libexec/ibus-en
gine-simple
vaibhav+    4927  0.0  0.0   9152  2328 pts/1    S+   11:49   0:00 grep --color=auto en
gine
```

Supervisor shutdown and system showing no zombie processes.

---

## 4. Engineering Analysis

This project demonstrates how operating system mechanisms work together to provide process isolation and resource control.

Isolation: Process isolation is achieved using the clone() system call with the CLONE_NEWPID, CLONE_NEWUTS, and CLONE_NEWNS flags. Filesystem isolation is enforced by mounting a private /proc directory and executing chroot before execv-ing the child.

Supervisor & Lifecycle: When a container exits, the kernel sends SIGCHLD. Our asynchronous signal handler uses waitpid with the WNOHANG flag to instantly reap the exited child without blocking the supervisor's main event loop, preventing zombie processes.

IPC & Threads: The logging pipeline uses a shared ring buffer accessed by pipe-reading producer threads and a file-writing consumer thread. We synchronize access using pthread_mutex_t to prevent race conditions, and pthread_cond_t condition variables to efficiently block threads without spinning the CPU.

Kernel Enforcement: Memory limits are enforced via a custom Linux Kernel Module (LKM) because user-space polling is too slow to stop aggressive allocations. The module tracks Resident Set Size (RSS). Soft limits trigger a dmesg warning, while hard limits trigger a strict SIGKILL directly from the kernel to protect the host system.

---

## 5. Design Decisions and Tradeoffs

- Namespace / Isolation
  - Choice: `chroot`-based isolation

- Tradeoff: Not as strong as full namespaces
- Justification: Simpler and sufficient for demonstration

---

### 🔹 Supervisor Design

- Choice: Single supervisor process
- Tradeoff: Single point of failure
- Justification: Centralized control simplifies management

---

### 🔹 Logging System

- Choice: Bounded buffer
- Tradeoff: Limited buffer size
- Justification: Prevents overflow and ensures synchronization

---

### 🔹 Kernel Monitor

- Choice: Timer-based RSS checking
- Tradeoff: Not real-time
- Justification: Low overhead and sufficient accuracy

---

### 🔹 Scheduling Experiment

- Choice: Linux nice values
- Tradeoff: Limited control over scheduler
- Justification: Demonstrates priority differences effectively

---

## 6. Scheduler Experiment Results

🔹 Setup Two CPU-bound workloads (cpu_hog) were executed concurrently on the VirtualBox VM.

```
prio_high was launched with a nice value of -20.

prio_low was launched with a nice value of 19.
```

🔹 Observations & Analysis Despite the massive disparity in scheduling priority, top output revealed that both processes consistently maintained ~100% CPU utilization. The high-priority process did not starve the low-priority process.

Why did this happen? This result perfectly illustrates the load-balancing mechanisms of the Linux Completely Fair Scheduler (CFS) on multi-core systems. Because the host VirtualBox VM has multiple CPU cores allocated to it, the CFS recognized that it did not need to force the two processes to compete for a single CPU's timeslices. Instead, the scheduler assigned prio_high to Core 0 and prio_low to Core 1. This satisfies the CFS's

goal of maximizing total system throughput while maintaining fairness, proving that nice values only force strict preemption when tasks are forced to contend for the same physical core.

---

## 7. Conclusion

This project successfully demonstrates:

- A working container runtime
- Kernel-level memory enforcement
- Logging pipeline with synchronization
- Multi-container supervision
- Scheduling behavior in Linux

The system integrates user-space and kernel-space components to showcase core operating system principles.